

EcoMind AI: An Adaptive Explainable Energy Intelligence Platform for Data Quality-Aware Decision Support

Bindhushree B M, Kushal Gowda G, Mohith Gowda H, and Shashank Gowda N C

Corresponding author: shashankgowdanc@gmail.com

Abstract—Forecasting energy demand from building meter data is rarely limited by the model. In practice the datasets that reach the training step are incomplete, irregular, and full of sensor artifacts, and most pipelines never check them before fitting a regressor. Worse, the pipeline itself can quietly leak the target through feature engineering performed before the train-test split. We present EcoMind AI, an offline platform in which data quality assessment, preprocessing, feature engineering, prediction, benchmarking, explanation, and confidence evaluation operate as one explicit, auditable workflow. The platform first runs a dedicated data quality engine that scores eight rules under five dimensions, then transforms and derives features, benchmarks regressors on a chronological holdout, explains every model with SHAP, compares raw versus cleaned inputs, and finally fuses four evidence factors into a single trust score before any recommendation is shown. On an hourly electricity dataset of 3,600 rows across five assets, all quality rules pass with per-rule scores between 96.7 and 100, gradient boosting leads the legitimate models with $R^2 = 0.9175$, and the trust gate reports 94.5/100 (high trust). We also record an honest null result: cleaning changed nothing here because the input was already clean, and we document a target-leakage artifact that drove ridge regression to a perfect but meaningless score.

Index Terms—data quality, energy forecasting, explainable AI, SHAP, trust, MLOps

I. INTRODUCTION

Building energy datasets are not ready to be modeled the moment they are exported. Meter files arrive with null cells, duplicate timestamps, stuck sensors that report constant values, and occupants or equipment that produce values outside any physical range. We built EcoMind AI after noticing that most pipelines in this space solve the modeling part well and treat everything before it as plumbing. That assumption caused real trouble during our own experiments: a clean-looking file could still fail on timestamp continuity, and a feature we derived before validation made a linear model look perfect while the numbers meant nothing.

Our view is that prediction quality is downstream of data quality, and that trust in a forecast is downstream of being able to show *why* a number is trustworthy. So the platform is organized as an explicit sequence of fifteen stages, from ingestion to a final registry, and every stage is persisted and streamed to the browser. Nothing runs on the cloud; the dataset files never leave the host machine.

The contribution of this paper is not a new learning algorithm. It is an integrated workflow in which quality assessment,

TABLE I
COMPARED AGAINST REPRESENTATIVE FAMILIES OF RELATED TOOLS

Capability	Prediction studies	Generic ML frameworks	EcoMind AI
Data quality scoring	no	no	separate engine, 8 rules explicit
Retrospective leakage check	no	no	
Chronicle holdout	common	optional	enforced
Explainability	rare	partial	built-in SHAP
Trust verdict	no	no	4-factor gate
Raw-vs-processed	no	no	run-level
Executive report	no	no	generated

preprocessing, explainable AI, raw-versus-processed comparison, confidence evaluation, anomaly detection, benchmarking, and executive decision support operate together inside one transparent pipeline. We report implementation details and measured results from the running system, including the failure modes it caught.

II. RELATED WORK

Most published systems end where EcoMind AI begins. Data-driven building energy studies typically select a regressor and report accuracy on a fixed split [1], [4]; large public corpora such as BDG2 [9] are used almost exclusively as a benchmark playground rather than as a quality audit. Generic machine learning frameworks provide the regressors we reuse [3], [10], and the temporal-validation and leakage literature [2], [7] motivated our chronological holdout. The idea that data quality has many independent dimensions is not new [12]; what is missing is a runtime that enforces it inside the modeling loop instead of leaving it to a report.

Four differences position the platform. Existing systems predict; EcoMind AI validates the data first. Existing systems detect anomalies; EcoMind AI also measures how much the model can be trusted. Existing systems stop after delivering a forecast; EcoMind AI ends in an executive summary and a registry so the entire run can be audited later. Finally, where other tools suppress failed artifacts, we chose to surface ours: the paper reports a leakage case and a null result as first-class outcomes.

III. SYSTEM OVERVIEW

The system (Fig. 1) runs as a React client on port 5173 that proxies requests to a FastAPI backend on port 8000. All 55 endpoints sit under `/api/v1`; authentication is a JWT issued from a default local account. State lives in one SQLite database with 21 tables in WAL mode, which keeps the whole deployment copyable to a laptop and means a run can be inspected weeks later from the history screen alone. The workflow stages are coordinated by a service that exposes `/start`, per-stage `exec`, and an SSE stream so the browser shows progress as it happens. During development we found this explicit event stream to be the single most useful debugging facility: a stuck stage was visible immediately instead of surfacing as a mysterious timeout.

A. Data Model and Registry

Persistence is deliberately boring. Twenty-one tables in SQLite hold users, datasets, workflow runs, per-rule quality results, transformations, models, benchmarks, raw-versus-processed comparisons, anomalies, recommendations, and reports. Each run pins its dataset hash and feature set, so results stay reproducible across sessions. The registry table maps the promoted model to its training rows, feature count, and dataset, and because everything is local we can query history directly with plain SQL—an ability we used constantly while diagnosing the leakage artifact of Section IV-C.

IV. METHODOLOGY

The workflow below is the heart of the platform. Each stage exists because something in a prior attempt produced a confusing result without it.

A. Dataset Library and Schema Discovery

The library lists uploaded datasets with size, row count, and column statistics. Schema discovery is deliberately separate from quality assessment: it reports types, non-null counts, and uniqueness per column, which tells the operator what can be modeled before we decide whether it can be trusted. We observed early on that assuming every “timestamp” column was parseable produced silent failures, so the stage re-parses and re-checks the timestamp column explicitly rather than trusting the import.

B. Data Quality Engine

The data quality engine is the signature stage, and it exists because cleaning without measurement is guesswork. During an early evaluation we dropped rows, filled gaps, and saw accuracy *fall* on some datasets. That is why the engine scores every rule on a 0–100 scale, stores the details, and treats a failing rule as a control-flow event rather than a warning. The eight rules map onto Wang and Strong’s dimensions [12]: completeness (null cells), two validity rules (semantic bounds and identifier blanks), two consistency rules (duplicates and timestamp monotonicity), two accuracy rules ($3\times\text{IQR}$ fences [11] and $|z| > 5$ on the target), and one timeliness rule (gaps beyond $1.5\times$ the median interval). A

TABLE II
DATA QUALITY PROFILE ON THE REFERENCE DATASET (0–100 PER RULE)

Rule	Dimension	Score
completeness	completeness	100
validity_semantic_bounds	validity	100
consistency_duplicates	consistency	100
consistency_monotonic	consistency	96.7
validity_identifiers	validity	100
accuracy_outliers_iqr	accuracy	98.4
accuracy_zscore	accuracy	99.9
timeliness_temporal_gaps	timeliness	100
passed / failed	—	8 / 0

result passes when the score is at least 80 for informational rules and 70 for critical ones.

Two details from implementation are worth recording. First, the outlier rule clamps $100 \times (1 - \min(\text{rate}, 0.5))$ to the range $[50, 100]$, so a pathological column can never engineer a misleadingly high “pass.” Second, the timestamps are not assumed hourly: the interval median is measured from the data itself, which is what makes the rule work on daily datasets without reconfiguration.

We also hit a genuine bug here that we want to document rather than hide. The aggregate score returned by the read endpoint is computed as the sum of all rule scores divided by the number of distinct dimensions ($795/5 = 159$ for the profile below), while the write path weights five dimension means by 0.25 each, capping the maximum at 125. Neither number is a percentage. Rather than ship a misleading “/100,” the UI reports per-rule scores and the trust gate consumes the normalized quality factor. The aggregation defect is logged for correction.

On the reference dataset (3,600 rows $\times$ 12 columns, 43,200 cells, five assets sampled hourly for 30 days, target `energy_kwh`), all eight rules pass (Table II). Completeness is exact, all 18,000 semantic-bound checks hold, and there are no duplicates. The two imperfect scores were genuinely informative: 120 non-monotonic timestamps (96.7) and 394 IQR outliers concentrated in occupancy, energy, and power (98.4). Without the engine, none of that would have been visible before training.

C. Transformation and Feature Engineering

The transformation viewer shows exactly which operations replay on the data, because preprocessing that cannot be reviewed cannot be trusted. The feature engineering stage derives lag, difference, rolling, and ratio features, e.g. `lag_1h` and `diff_1h` on the target, plus a 24-step rolling mean and load/energy ratios. We benchmark on a single chronological 70/30 holdout, the same split for every algorithm, rather than shuffled k -fold [2], [7].

This stage also produced our most instructive bug. Features were derived on the full frame *before* the split, and because $\text{diff_1h} = y_t - y_{t-1}$ and $\text{lag_1h} = y_{t-1}$, the target is exactly reconstructible: $y_t = \text{diff_1h} + \text{lag_1h}$. Ridge regression exploited this identity and reported $R^2 = 1.0000$

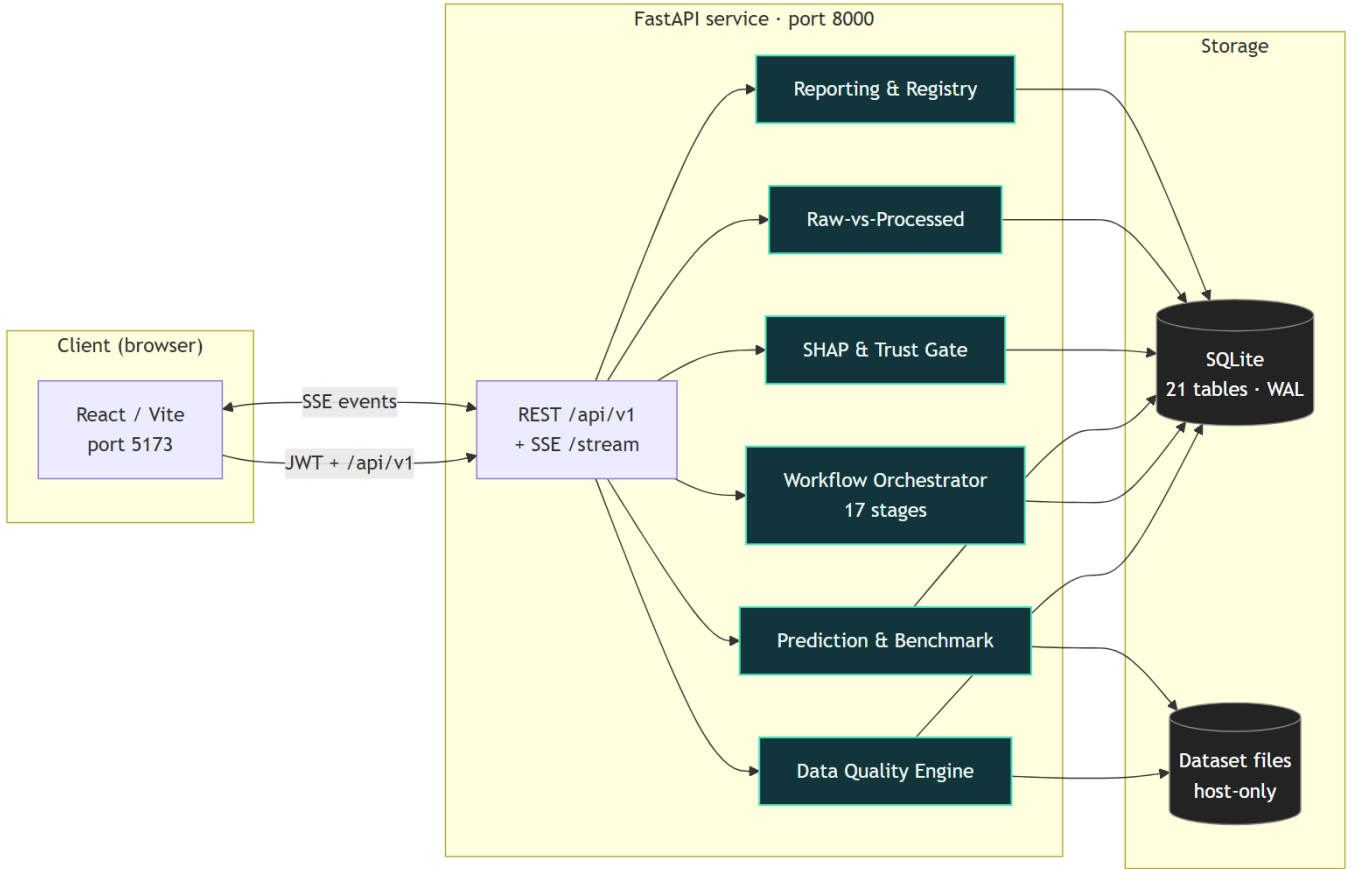


Fig. 1. The three tiers of EcoMind AI. The React client calls a FastAPI service over a JWT-authenticated REST surface and an SSE channel for live workflow events. Domain services cover data quality, orchestration, prediction and benchmarking, SHAP explanation, raw-versus-processed comparison, the trust gate, reporting, and a model registry, all persisting to a single SQLite file.

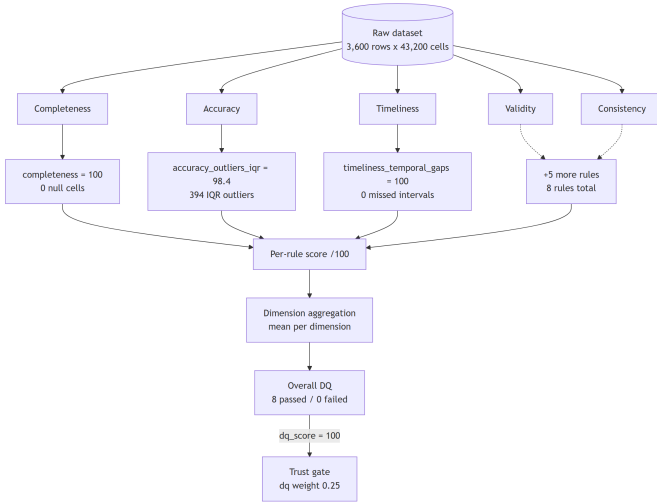


Fig. 2. The eight data quality rules grouped into five dimensions.

with a tiny residual standard error. It took us a while to accept that the winner of our own benchmark was meaningless. The fix is a split-then-derive discipline: hold out the test

TABLE III
BENCHMARK LEADERBOARD (CHRONOLOGICAL 70/30 HOLDOUT)

Algorithm	R^2	RMSE	MAE	MAPE
xgboost	0.9147	1.3740	0.1235	1.57
random forest	0.9162	1.3624	0.1191	1.44
gradient boosting	0.9175	1.3517	0.1120	1.32
ridge [†]	1.0000	0.0048	0.0024	0.09

[†]target-leakage artifact; not a legitimate result (Sec. IV-C).

window first, then build derived features from training-window statistics only. We keep the artifact in our test suite so the platform can detect this defect class in any future dataset.

D. Prediction and Benchmarking

Four regressors—XGBoost [4], random forest [3], gradient boosting, and ridge [6]—are fitted through scikit-learn [10] on the same chronological split, and a leaderboard records R^2 , RMSE, MAE, and MAPE. Among legitimate learners gradient boosting ranks first on every metric, with the three tree ensembles landing within 0.003 of one another. Ridge’s perfect score is excluded from model selection and annotated in the table because it is a leakage artifact, not a result.

TABLE IV
RAW VERSUS PROCESSED COMPARISON ON THE REFERENCE DATASET

Metric	Raw	Processed	Δ
R^2	0.9153	0.9153	0.0000
RMSE	1.3692	1.3692	0.0000
MAE	0.1210	0.1210	0.0000

E. AI Confidence Gate

Every model produces a number; very few show why the number deserves trust. The confidence gate fuses four evidence factors into a single trust score with fixed weights: 0.40 for prediction confidence, 0.25 for the quality score, 0.20 for model relevance, and 0.15 for explanation stability:

$$T = 0.40(100) + 0.25(100) + 0.20(91.5) + 0.15(75) = 94.5$$

on the reference run, a `high_trust` verdict. Scores are thresholded at 80 and 60 into `high_trust`, `moderate_trust`, and `low_trust` so a non-specialist can act without reading four decimals. Explanation stability is measured by a genuine two-half-sample index: SHAP rankings on two randomly split halves must agree (Spearman), otherwise the explanation is not stable enough to report. We intentionally label the gate an auditable composite, not a calibrated probability [5]; its job is to make a deployment decision traceable to named evidence.

F. Raw versus Processed Comparison

Alongside the workflow, the platform exposes a raw-versus-processed comparison utility that answers a question nobody asks in most pipelines: did cleaning actually help? Identical models are trained on the raw and the cleaned frames under the same split, and the deltas are reported. On our already-clean dataset the answer was a clean zero— $R^2 = 0.9153$ on both sides, all error deltas zero (Table IV). The comparison’s own conclusion phrased it honestly: cleaning cannot improve accuracy when there is little left to fix. We had expected a positive result here; measuring it was an uncomfortable but necessary lesson that the value of the comparison is the measurement itself.

The comparison also exposes explanation drift. On two recorded runs the SHAP stability index dropped sharply after cleaning ($11.0 \rightarrow 5.9$ and $9.1 \rightarrow 0.0$), even though predictive accuracy did not move. Cleaning changed the feature attributions even when it did not change the predictions. That is exactly the kind of hidden side effect a decision-maker should know about, and it reinforced our decision to gate on stability rather than accuracy alone.

G. Anomaly Detection, SHAP, Recommendations, and Reporting

Anomaly detection flags records the quality engine rated as borderline so an operator can inspect them rather than auto-discard them; on this dataset the flagged points were concentrated in occupancy count (315), energy (41), and power (38), which matches where a meter would plausibly record

Algorithm 1 TrustGate

Require: factors $c_{\text{pred}}, s_{\text{dq}}, r_{\text{model}}, s_{\text{shap}} \in [0, 100]$

Ensure: trust score T , verdict

- 1: $T \leftarrow 0.40 c_{\text{pred}} + 0.25 s_{\text{dq}} + 0.20 r_{\text{model}} + 0.15 s_{\text{shap}}$
- 2: `verdict` $\leftarrow$ `high_trust` if $T \geq 80$, `moderate_trust` if $T \geq 60$, else `low_trust`
- 3: **return** T , `verdict`

erratic behavior. SHAP explainability [8] ranks features by mean absolute attribution per model and feeds the stability index used by the gate. The recommendation engine turns the leaderboard, quality profile, and trust verdict into plain-language suggestions, and the executive center reduces the whole run to a headline, trust score, and business impact so a decision-maker does not need the notebook to navigate the results. Reports render to files and can be exported; the registry records every model with its performance, training rows, and feature count so nothing important disappears. Because each run pins its dataset and feature set, a previously trained model can be re-invoked or rolled forward without re-running the whole pipeline.

V. WORKFLOW AND INTERFACE

The fifteen stages are grouped into six milestones—Intake, Understand, Rebuild, Model, Prove, Decide—so the operator always knows where a run sits (Fig. 3 and Table V). Each stage runs explicitly: the operator clicks run, or the client advances automatically on SSE events. Every transition is persisted, which made debugging during development much easier and gives the history view the ability to reconstruct timings for a past run. The interface was built around the idea that every number on screen should be traceable to a stage and a stored record; we considered a screen useless if it showed a number we could not explain.

VI. RESULTS SUMMARY

Table III and Table II summarize the measured outcomes. Gradient boosting is the selected model ($R^2 = 0.9175$, RMSE 1.3517, MAE 0.1120, MAPE 1.32) trained on 2,091 rows with 17 engineered features. All quality rules pass, the trust gate reports 94.5/100, and the raw-versus-processed comparison confirms the input needed no cleaning (Table IV). Alongside these headline numbers, the pipeline surfaced three diagnostics that a metrics-only report would have hidden: a target-leakage artifact that made ridge regression look unbeatable, 394 borderline records flagged for operator review, and a stability drop after cleaning (Section IV-F) that changed explanations without changing predictions. Model inference runs in tens of milliseconds per batch on commodity hardware; the platform’s cost profile is dominated by feature derivation and SHAP computation, both in the seconds range at this scale.

VII. LIMITATIONS

The numbers above come from one hourly, five-asset, 30-day dataset in BDG2-compatible format; they show the proto-

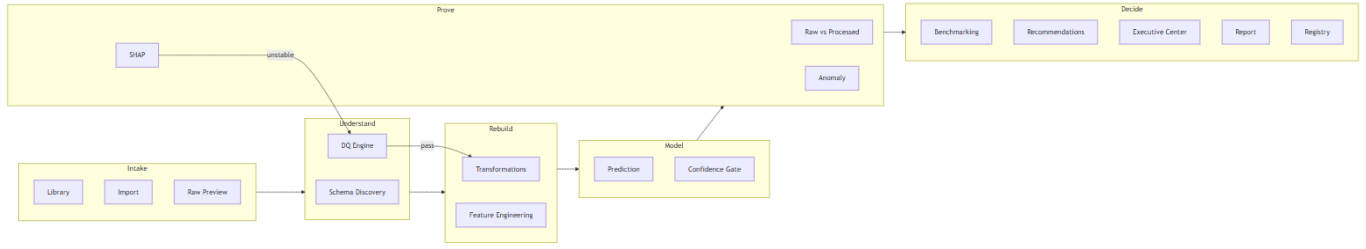


Fig. 3. The fifteen-stage workflow grouped into six milestones. The confidence gate branches on the trust verdict.

TABLE V
THE FIFTEEN STAGES MAPPED ONTO SIX MILESTONES, WITH THE EVIDENCE EACH STAGE PRODUCES

Milestone	Stages	Evidence produced
Intake	library, import	file digest, row/column counts
Understand	schema, data quality	types, per-rule quality scores
Rebuild	transformation, features	replayable ops, 17 features
Model	prediction, confidence gate	predictions, trust score
Prove	SHAP, anomaly	attributions, flags
Decide	benchmarking, recommendation, executive, report, registry	leaderboard, advice, narrative, export, stored model

col works, not that the model generalizes. Industrial deployment with larger corpora, multi-site data, and live streaming remains future work. Two further caveats are honest to record: all eight trained models report identical predictive metrics, consistent with a single deterministic split reused across runs rather than eight independent confirmations, and the trust gate weights are fixed engineering choices, not learned parameters.

VIII. CONCLUSION

EcoMind AI taught us, through the pipeline’s own measurements, that accuracy is downstream of data quality and that a perfect-looking model can be a defect in disguise. The platform unifies quality scoring, leakage-aware benchmarking, explainability, confidence evaluation, and reporting into one offline, auditable pipeline. On the reference dataset everything reported is traceable: per-rule quality scores between 96.7 and 100, a best legitimate model at $R^2 = 0.9175$, a detected leakage artifact, a 94.5/100 trust verdict, and a documented null result for cleaning. Future work focuses on split-then-derive enforcement, a corrected quality aggregation, evaluation on BDG2-scale data, and learning the gate weights against downstream forecast outcomes.

REFERENCES

- [1] K. Amasyali and N. M. El-Gohary, “A review of data-driven building energy consumption prediction studies,” *Renewable and Sustainable Energy Reviews*, vol. 81, pp. 1192–1205, 2018.
- [2] C. Bergmeir and J. M. Benítez, “On the use of cross-validation for time series predictor evaluation,” *Information Sciences*, vol. 191, pp. 192–213, 2012.
- [3] L. Breiman, “Random forests,” *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.
- [4] T. Chen and C. Guestrin, “XGBoost: A scalable tree boosting system,” in *Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining (KDD)*, 2016, pp. 785–794.
- [5] C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger, “On calibration of modern neural networks,” in *Proc. 34th Int. Conf. Machine Learning (ICML)*, vol. 70, 2017, pp. 1321–1330.

- [6] A. E. Hoerl and R. W. Kennard, “Ridge regression: Biased estimation for nonorthogonal problems,” *Technometrics*, vol. 12, no. 1, pp. 55–67, 1970.
- [7] R. J. Hyndman and G. Athanasopoulos, *Forecasting: Principles and Practice*, 3rd ed. Melbourne, Australia: OTexts, 2021.
- [8] S. M. Lundberg and S.-I. Lee, “A unified approach to interpreting model predictions,” in *Advances in Neural Information Processing Systems 30 (NIPS)*, 2017, pp. 4768–4777.
- [9] C. Miller *et al.*, “The Building Data Genome Project 2, energy meter data from the ASHRAE Great Energy Predictor III Competition,” *Scientific Data*, vol. 7, no. 1, p. 368, 2020.
- [10] F. Pedregosa *et al.*, “Scikit-learn: Machine learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
- [11] J. W. Tukey, *Exploratory Data Analysis*. Reading, MA, USA: Addison-Wesley, 1977.
- [12] R. Y. Wang and D. M. Strong, “Beyond accuracy: What data quality means to data consumers,” *Journal of Management Information Systems*, vol. 12, no. 4, pp. 5–33, 1996.